

ESTRUTURA DE DADOS II – Entregar na data da prova

1. Explique o funcionamento do algoritmo Bubble Sort.
2. Qual a complexidade dos algoritmos bubble sort, selection sort, insertion sort e quick sort. Qual tem melhor desempenho? Explique
3. Dos algoritmos de ordenação de complexidade quadrática, qual tende a ter um melhor desempenho? Justifique.
4. Dentre todos algoritmos de ordenação eficientes, qual possui o melhor desempenho para uma ampla variedade de situações?
5. Cite 3 algoritmos de ordenação de complexidade $O(n \log n)$.
6. Se a quantidade de comparações do Bubble Sort e do Selection Sort são iguais, por que a medida de tempo dos dois é diferente. Explique.
7. Explique o algoritmo de ordenação por contagem, counting sort.
8. Dado um vetor desordenado de 131072 posições, quantos itens serão examinados no máximo para que ele encontre um determinado valor no vetor ou retorne que o valor não existe?
9. Qual método de ordenação é apresentado abaixo?

```

1 void X ( int v[] , int n) {
2     int i , j;
3     int aux ;
4     for (i = 0; i < n -1; i ++) {
5         for (j = 1; j < n -i; j ++) {
6             if ( v[j] < v[j -1] ) {
7                 aux = v[j] ;
8                 v[j] = v[j -1];
9                 v[j -1] = aux ;
10            } } } }

```

10. Como o algoritmo acima poderia ser melhorado?
11. Observando a figura abaixo, explique o funcionamento do método de ordenação isort.

i	u_1	u_2	u_3	u_4	u_5	u_6
1	34	17	68	29	50	47
2	17	34	68	29	50	47
3	17	34	68	29	50	47
4	17	29	34	68	50	47
5	17	29	34	50	68	47
	17	29	34	47	50	68

12. Observe o algoritmo representado a seguir:

```
static int metodo(int[] v, int inicio, int fim, int e) {  
    if (inicio > fim) {  
        return -1;  
    }  
    int i = (inicio + fim) / 2;  
    if (v[i] == e) {  
        return i;  
    } else if (v[i] < e) {  
        return metodo(v, i + 1, fim, e);  
    } else {  
        return metodo(v, inicio, i - 1, e);  
    }  
}
```

Esse algoritmo está processando a operação:

- A. busca sequencial em um vetor V[]
- B. busca binária em um vetor ordenado V[]
- C. ordenação do vetor V[] pelo método rápido (QUICKSORT)
- D. ordenação do vetor V[] pelo método da intercalação (MERGESORT)

13. O MergeSort é considerado um algoritmo estável. Isso significa que:

- a) Ele não altera a ordem relativa dos elementos iguais.
- b) Ele sempre usa a mesma quantidade de memória.
- c) Ele é mais rápido que o QuickSort no caso médio.
- d) Ele nunca precisa de memória adicional.

14. Explique as interfaces Comparable e Comparator em Java e como elas são utilizadas para realizar a ordenação de objetos.

15. Em que situações seria mais apropriado utilizar Comparable e em quais casos Comparator seria mais indicado?

16. Qual a diferença entre busca binária e busca sequencial?

17. Como a interface Comparator é usada para auxiliar na busca binária e por que é importante que a lista esteja ordenada antes de realizar essa busca?

18. Simule o funcionamento do algoritmo de busca binária para determinar se os itens 33, 50, 90 constam do vetor L = {10, 16, 27, 31, 33, 37, 41, 49, 53, 57, 68, 69, 72, 77, 84, 89, 95, 99}.

ENVIAR CÓDIGO (link do git)

Exercício – Ordenação de aprovados no vestibular

Uma instituição de ensino precisa organizar a lista de candidatos aprovados em seu processo seletivo. Cada candidato possui nome e nota final obtida no vestibular. Link do csv:

<https://drive.google.com/file/d/1OsUFFdjyvpPIO6alXDUMiHuwgetSP0kr/view?usp=sharing>

Implemente uma solução em Java utilizando a classe genérica `Sorts<T>` e o algoritmo de ordenação estudado em aula.

Requisitos

1. Crie uma classe `Candidato` contendo, no mínimo:
 - o nome;
 - o nota.
2. A classe `Candidato` deverá definir seu critério padrão de comparação, considerando os seguintes critérios, nesta ordem:
 - o primeiro, nota em ordem decrescente;
 - o em caso de empate na nota, nome em ordem alfabética crescente.
3. Carregue os dados do csv
4. Apresente na tela a lista antes da ordenação.
5. Utilize a classe genérica `Sorts<T>` para realizar a ordenação. Não utilize `Arrays.sort()`, `Collections.sort()` ou métodos equivalentes.
6. Após a ordenação, apresente a lista de candidatos classificados.
7. A ordenação deverá produzir um resultado semelhante a:
 1. Ana Paula - 98,7
 2. Carlos Henrique - 97,5
 3. Beatriz Silva - 97,5
 4. Daniel Souza - 96,8

...

Observe que, quando dois candidatos possuem a mesma nota, o candidato cujo nome vem primeiro alfabeticamente deverá aparecer antes.

Exemplo

Considere os candidatos:

Carlos - 85,5

Ana - 92,0

Bruno - 85,5

Daniel - 92,0

Após a ordenação, o resultado esperado é:

Ana - 92,0

Daniel - 92,0

Bruno - 85,5

Carlos - 85,5

A classe Candidato deve ser responsável por definir como dois candidatos devem ser comparados, enquanto a classe Sorts<T> deve permanecer genérica, sem conhecer os atributos nome ou nota.

Dessa forma, a solução deverá separar as responsabilidades:

- Candidato → define o critério de comparação;
- Sorts<T> → implementa o algoritmo de ordenação;
- Main → gera os dados, executa a ordenação e apresenta os resultados.